

ROBOT ARM



Introduction

The part of the project is on the arm control, crop recognition et crop picking. To do that, we will use the robot arm DofBot SE that we can find its specification on the annex file. Firstly, we will configure the environment of work, and after that we will read to see what the DofBot SE constructor gave us and how to use them. After that, we will attack the arm control.

Verification of the recognition of the robot arm by the computer:

-The stm32 card is a peripheric of the type of serial (UART).

Command to display it: `ls /dev/ttyUSB*`

```
kelly@kelly-HP-ProBook-445-14-inch-G10-Notebook-PC:~$ ls /dev/ttyUSB* /dev/ttyACM*
ls: impossible d'accéder à '/dev/ttyACM*': Aucun fichier ou dossier de ce nom
/dev/ttyUSB0
```

The card is recognized as **ttyUSB0**.

- The camera is not a peripheric of serial communication, it sends a flow of heavy digital data.

Command to display it: `ls /dev/video*`

Before branching the camera:

```
kelly@kelly-HP-ProBook-445-14-inch-G10-Notebook-PC:~$ ls /dev/video*
/dev/video0 /dev/video1
```

After branching the camera:

```
kelly@kelly-HP-ProBook-445-14-inch-G10-Notebook-PC:~$ ls /dev/video*
/dev/video0 /dev/video1 /dev/video2 /dev/video3
```

There are two names that appeared: **video2** and **video3**

This information will help us in the next to debug when some problem will occur.

Requirement

1- APP(IOS/Android):

- YahboomRobot.

It is the phone application for controlling the arm.

2- VM file

- VMware-workstation-full
- VM22.04_ROS2_DofBot-SE

They are the files needed to launch the virtual machine which contain the environment precisely made for the DofBot SE. There are the most important resources.

3- Instruction manual (no need for now)

It is a set of images to know how to manipulate the robot arm.

4- Hardware_3DModelFile

- Bus servo
- DOFBOT Hardware references Manual
- 3D Model file
- Stm32 board

They constitute the information about the hardware part.

5- PC_Software

- YahboomArmEn

This is the environment of simulation. It is a software made with the arm already spawned into it.

6- Code

- DofBotSE-Code-New

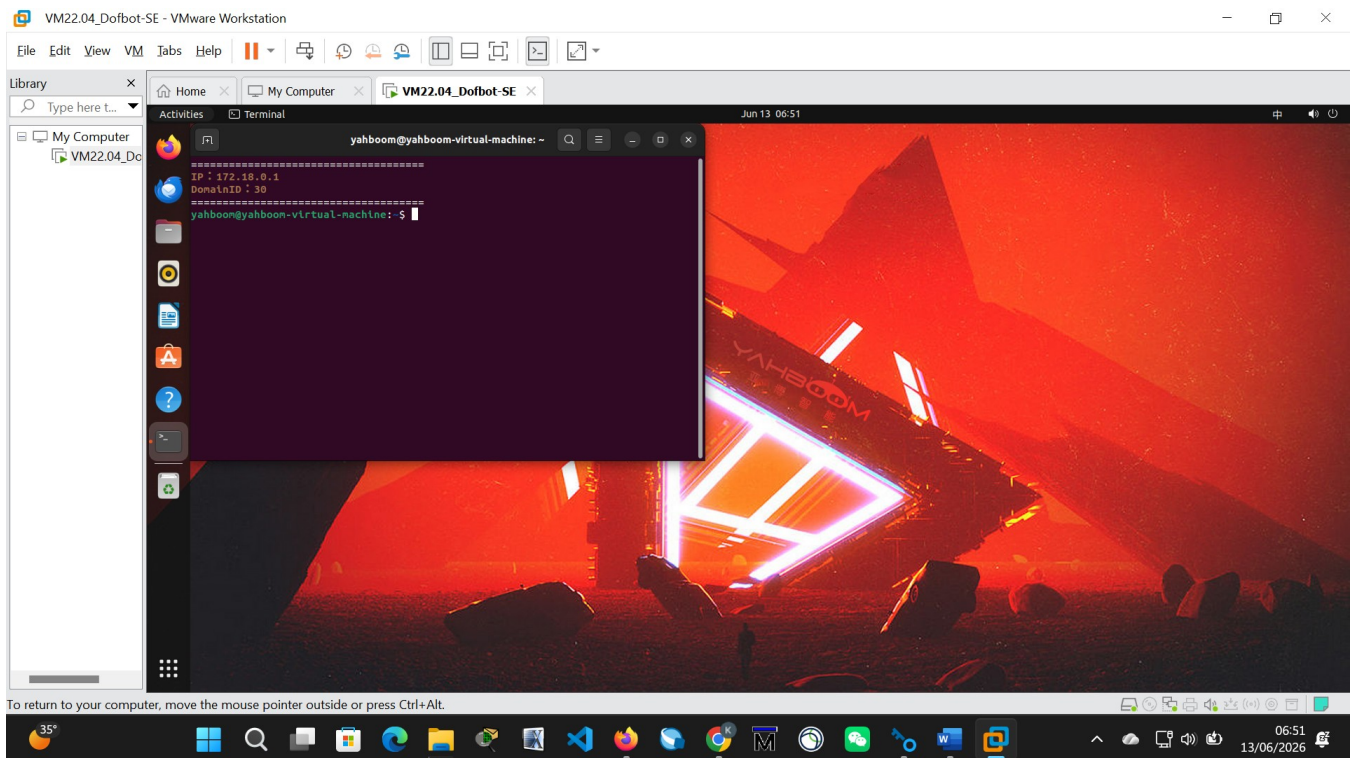
7- Communication Protocol-Python Library

- Install Python Module
- Stm32 and Virtual machine Serial port protocol
- MCU coprocessor I2C communication protocol
- PC APP remote control communication protocol

We did download everything the Yahboom website gave us, but the computer is not enabled to extract the VM22.04_ROS2_DofBot-SE.zip because of a lack of storage. To deal with this problem, we will use an external storage SSD.

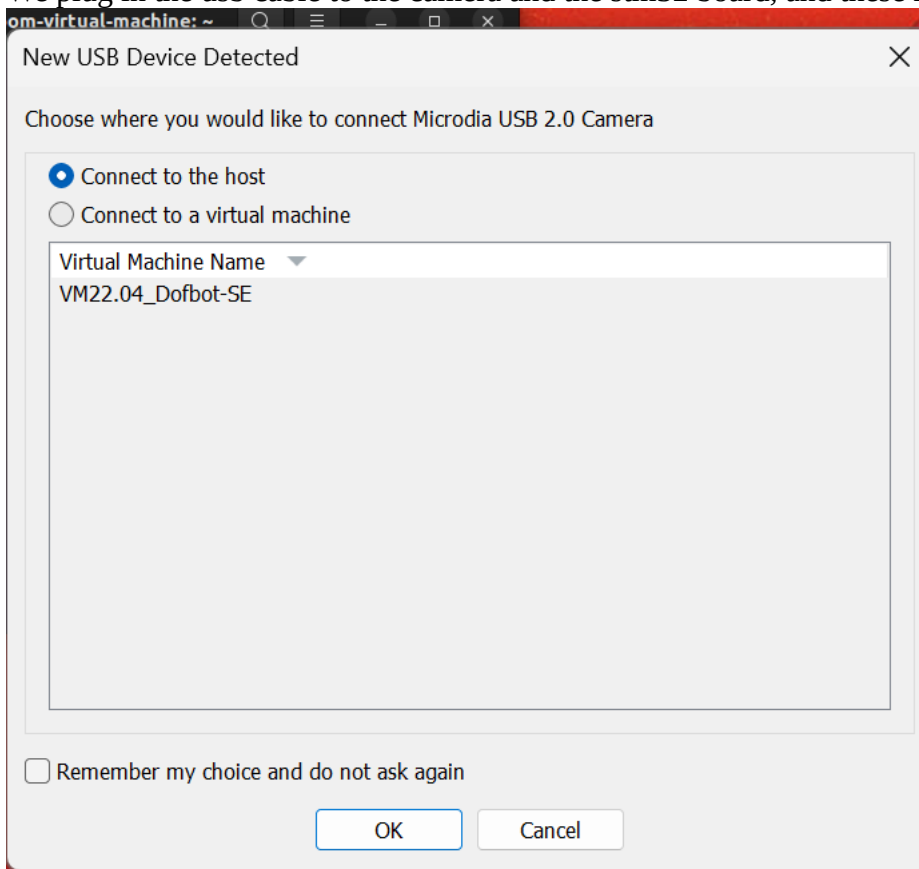
Problem of storage

After the installation of the software needed, we managed to get the virtual machine working.



Configuration to connect the robot arm to the virtual machine

We plug in the usb cable to the camera and the stm32 board, and these messages will appear:

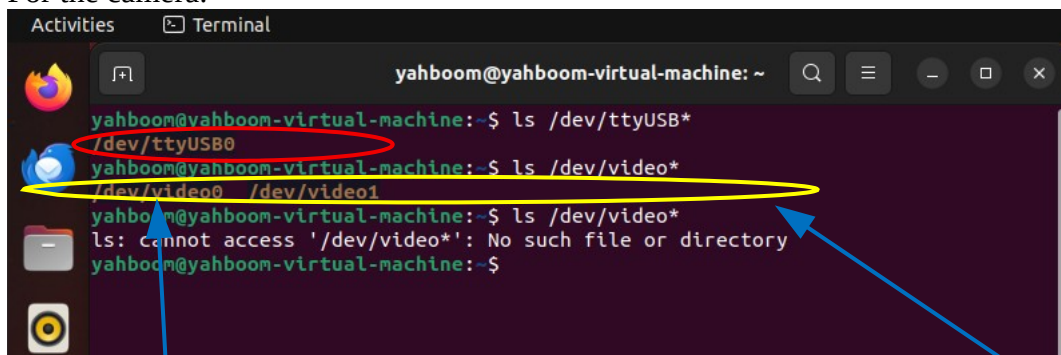


To connect the robot arm, choose “**connect to virtual machine**” and select **VM22.04-DOFBOT-SE**” or directly do in the option **VM** and select “**network adapter, Microdi USB 2.0 Camera and QinHeng USB Serial**” like on the picture bellow

Image here

To verify if the robot arm and the camera are detected in the virtual machine, we can display the usb port connected:

For the camera:



A terminal window titled 'Terminal' with the prompt 'yahboom@yahboom-virtual-machine: ~'. The terminal shows the following commands and output:

```
yahboom@yahboom-virtual-machine:~$ ls /dev/ttyUSB*  
/dev/ttyUSB0  
yahboom@yahboom-virtual-machine:~$ ls /dev/video*  
/dev/video0 /dev/video1  
yahboom@yahboom-virtual-machine:~$ ls /dev/video*  
ls: cannot access '/dev/video*': No such file or directory  
yahboom@yahboom-virtual-machine:~$
```

Annotations: A red circle highlights '/dev/ttyUSB0'. A yellow oval highlights '/dev/video0 /dev/video1'. A blue arrow points from the text 'With the camera' to the yellow oval. Another blue arrow points from the text 'With the camera unplugged' to the error message 'ls: cannot access '/dev/video*': No such file or directory'.

With the camera

With the camera unplugged

Connexion to the robot arm

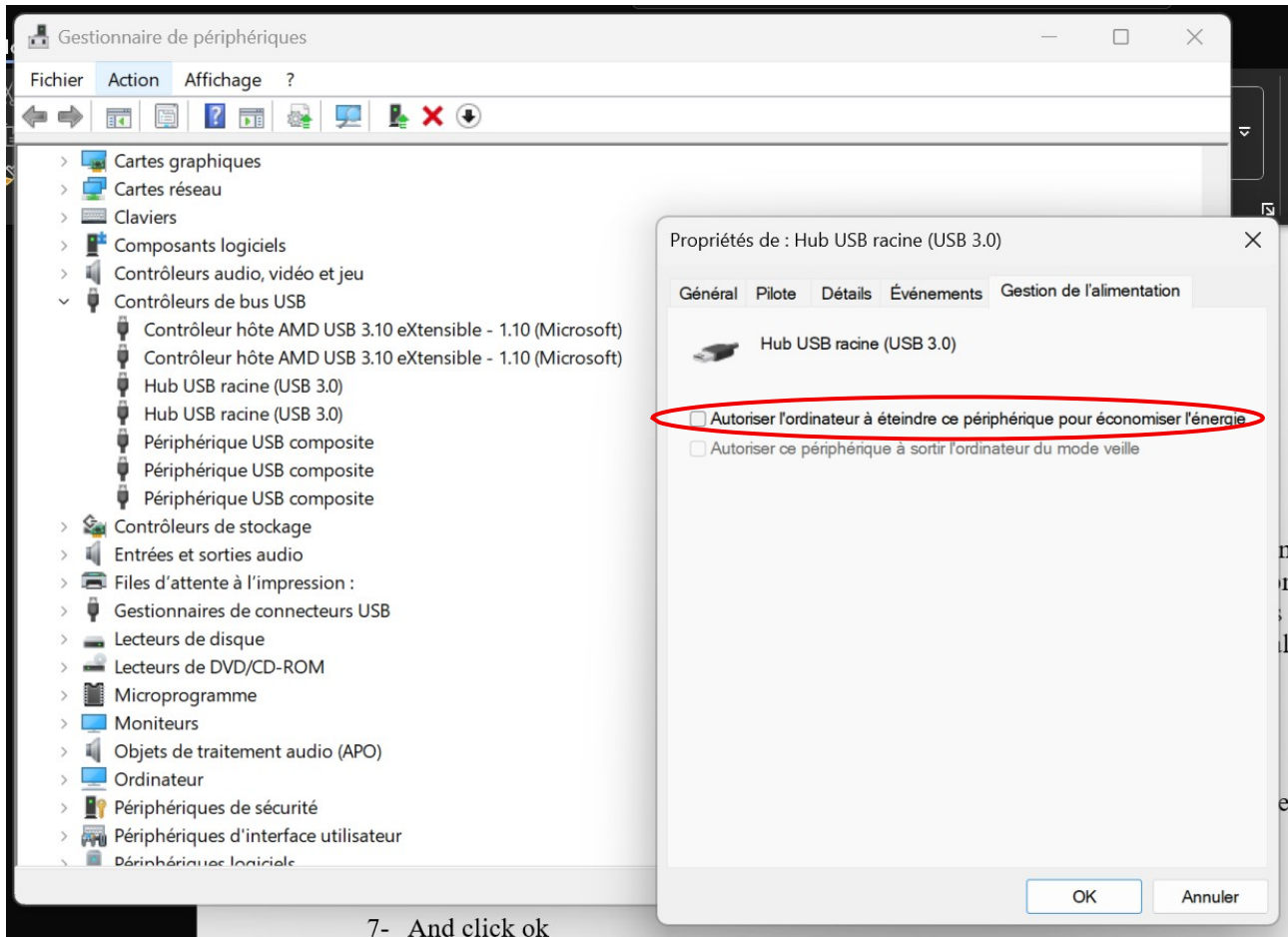
A problem appears when we try connecting the virtual machine workspace to the robot arm, here it is:

This problem is due to a windows IOS alimentation protection. When the VM is performing a resource-intensive task (such as loading ROS2 or running AI calculations), the drive consumes a spike in power. If your laptop's USB port doesn't provide enough power, or if Windows is trying to conserve energy, the drive “drops out” for a microsecond, abruptly cutting off the signal. To deal with this problem we must modify the Windows OS about the USB Hub.

To do so, we go in:

- 1- Device Manager.
- 2- Scroll down to the very bottom of the list until you see “USB Bus Controllers.”
- 3- Right-click on “USB Root Hub” (do this for each one if you have more than one)

- 4- Select Properties
- 5- Go to the Power Options tab
- 6- Uncheck the box: “Allow the computer to turn off this device to save power”
- 7- And click ok



7- And click ok

After doing this, the problem will be resolved.

Connection to the robot arm

After doing the configuration above, must now connect the virtual machine and the robot arm. To do so, will launch this command:

Our first command running give this:

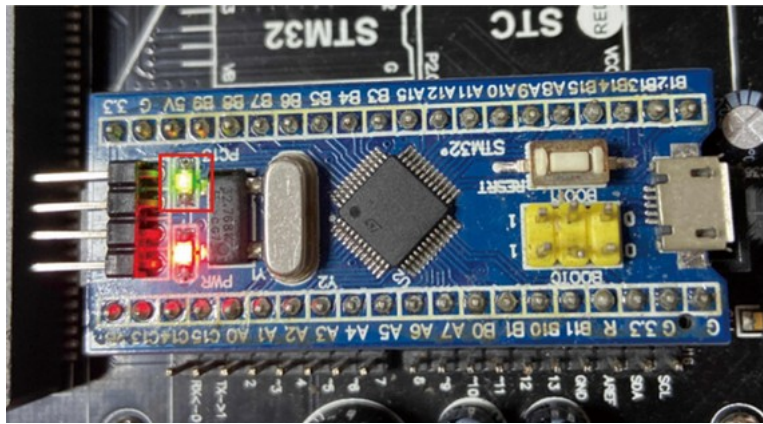
Our second command running give this:

These errors indicate that the virtual machine cannot connect to the robot arm with through the port /dev/myserial.

- 1- First clue: Verify the permission

All the permission needed to access the port are provided. So, the problem does not come from the permission.

Indeed, the Stm32's led is supposed to be green like in the manual, but mine is blue.



How it is supposed to be

What I have

In this case, Yahboom company give the solution of reflashing the Stm32 card. In the file given by Yahboom company, there is a file “template.hex” for reflashing the card.

Card reflash

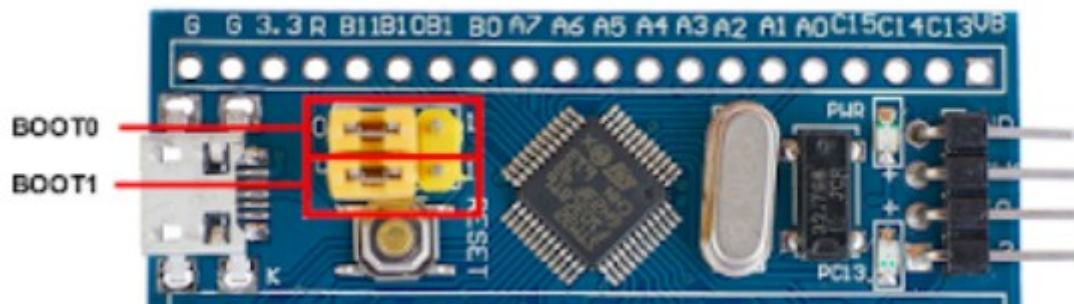
When we try to reflash the card, this error above appears. This error gave us important information about how to configure and establish the communication with the Stm32 card.

- **Intel Hex:** is the type of the file we must use to reflash the card
- **57600:** the speed of communication of the card, so to be able to communicate with the card, we must configure the virtual machine on the same speed. Indeed, in communication, the different devices must be synchronized and they must be on the same wavelength; otherwise, communication is not possible.
- **8:** the number of bits of a data (8 bit for one character)
- **E:** is the type of the parity, and in this case, it means Even
- **1:** the number of stop bit, then only one bit will be used to define the end of the transmission

To configure the virtual machine according to this information we use this command:

```
yahboom@yahboom-virtual-machine:~/Desktop/XDU_K/stm32flash-0.7$ stty -F /dev/ttyUSB0 57600 cs8 parenb -parodd -cstopb -echo
```

We must also put the jumper BOOT0 of the card in position HIGH (1). Here is what the jumper looks like:



The jumper in position HIGH indicates to the card that it must load the sending program in the flash part. It prepares the card for the reflash.

We tried again to reflash the card, but the same problem occurred. To deal with that problem, we installed stm32flash-0.7 on the website the link was given in the error below.

After installing the package, there was not the executable. Here are what it contained:

```
yahboom@yahboom-virtual-machine:~/Desktop/XDU_K/stm32flash-0.7$ ls
aclocal.m4      configure      init.h         missing        serial_w32.c
Android.mk      configure.ac   INSTALL        parsers        stm32.c
ar-lib          depcomp       install-sh     port.c         stm32flash.1
AUTHORS         dev_table.c   ltmain.sh     port.h         stm32.h
ChangeLog       gpl-2.0.txt   m4            protocol.txt   TODO
compile         HOWTO        main.c        serial_common.c  utils.c
compiler.h      i2c.c        Makefile      serial.h       utils.h
config.guess    I2C.txt      Makefile.am   serial_platform.c
config.sub      init.c       Makefile.in   serial_posix.c
```

Thanks to the Makefile, we could compile the file with make.

After the compilation, we could see that the executable was created as you can see in green.

Image here

The reflashing this time worked:

To conclude the configuration, we put the jumper BOOT0 in position 0 and reboot the card. After we launched the main program python3 ~/rosmaster/YahboomArm.pyc. Here is the result:

The main program was successfully launched. The robot, then changed its position.

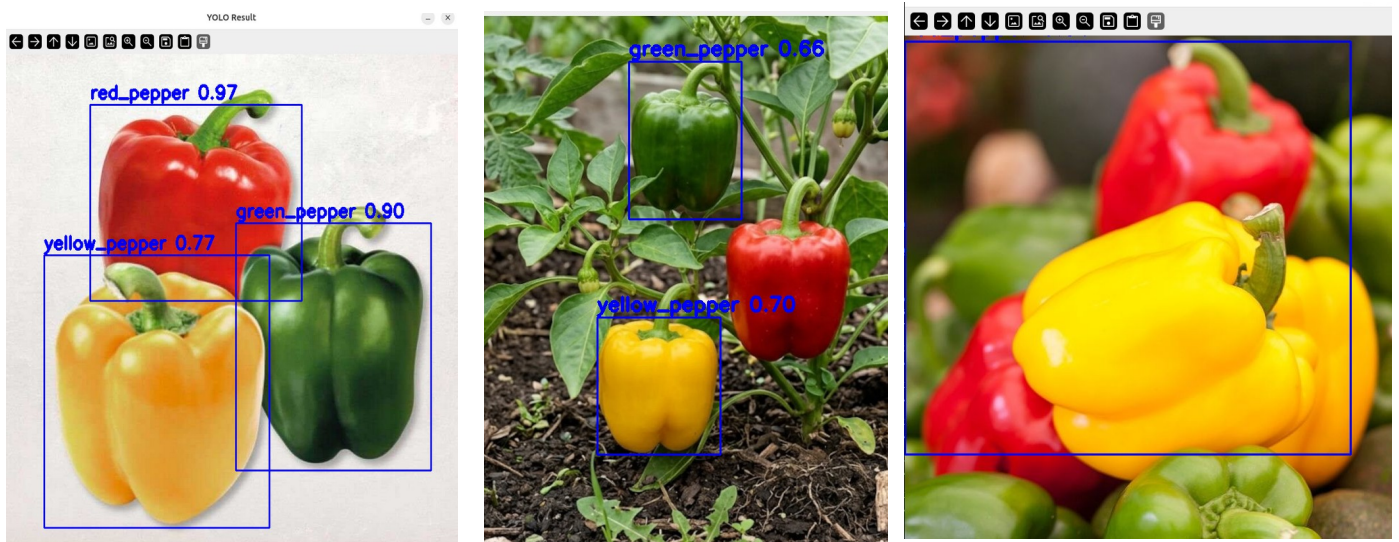
Position before the connection

Position after the connection

Crop recognition

For the crop recognition, we will use Yolo26, the last version of Yolo. Thanks to Yolo26, which quick in detection and can capture small details, we will be able to recognize the fruit and vegetables even if we have not enough details.

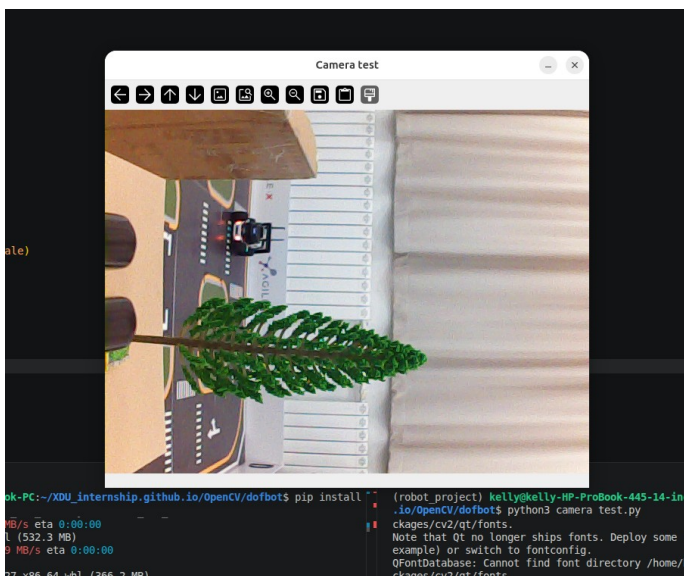
We chose green pepper, yellow pepper and red pepper for the first test. After training our model and deploying it, we downloaded new images of red, green and yellow pepper to test the vegetables recognition. And we got this



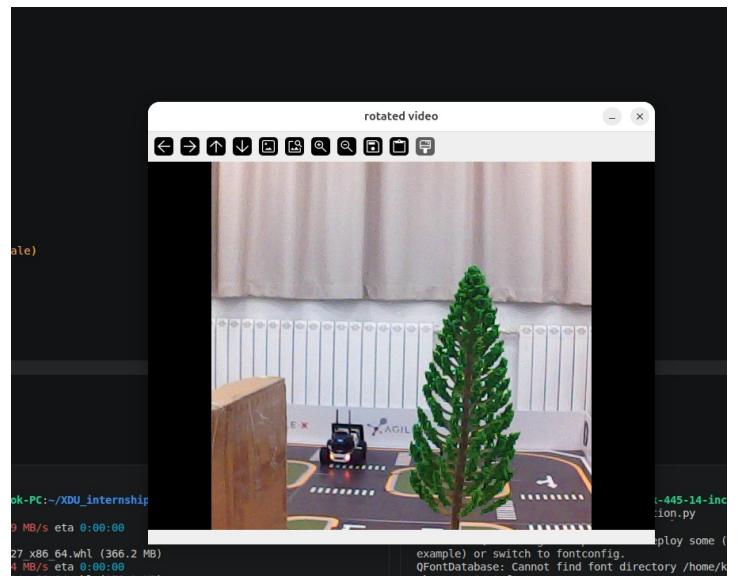
when can see that the vegetables recognition doesn't work very well, but it remain usable. To make it more efficient , we will train enough our model with more images.

Camera displaying problem

The video displayed by the camera is upside down, so to use it, we must put it in the reight position.



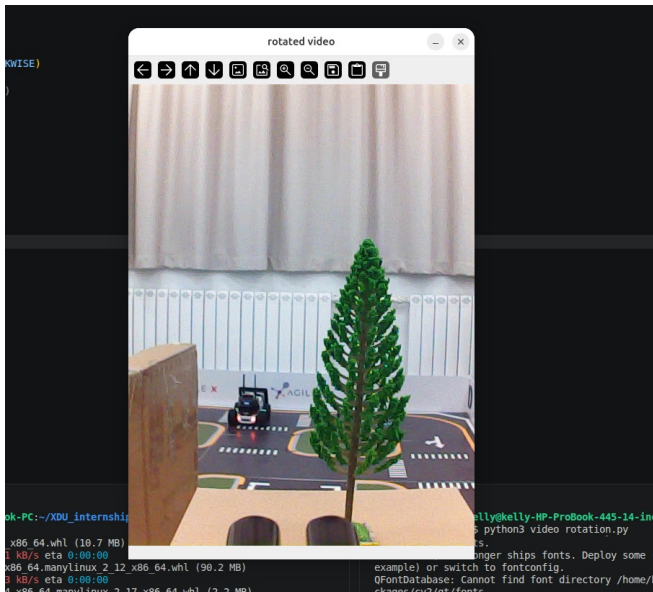
Before opencv process



After opencv process

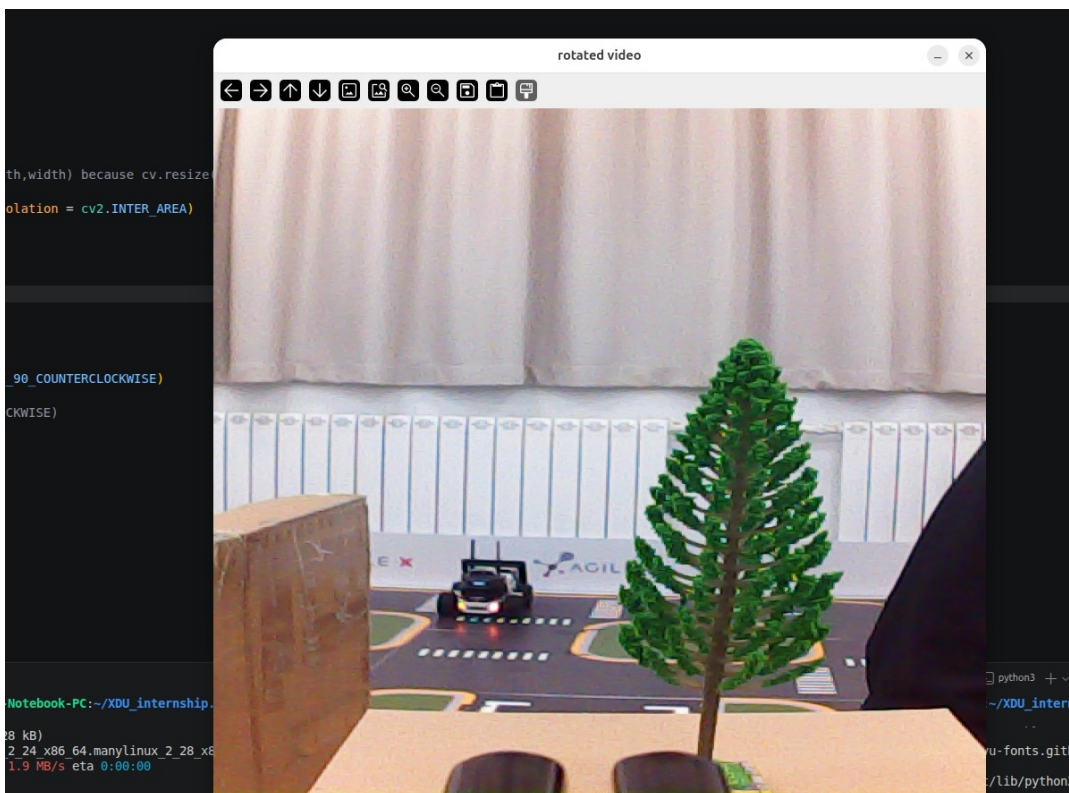
We have another problem, the video is in the right position but it has been zoomed in. Because of that the grapper has disappear, while we need to see it in the video to indicate to the arm how to pick the crop.

We change the algorithm and this give the following result:

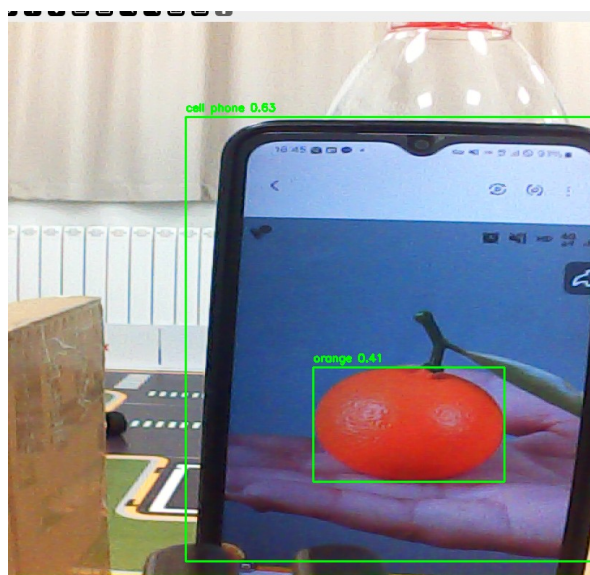
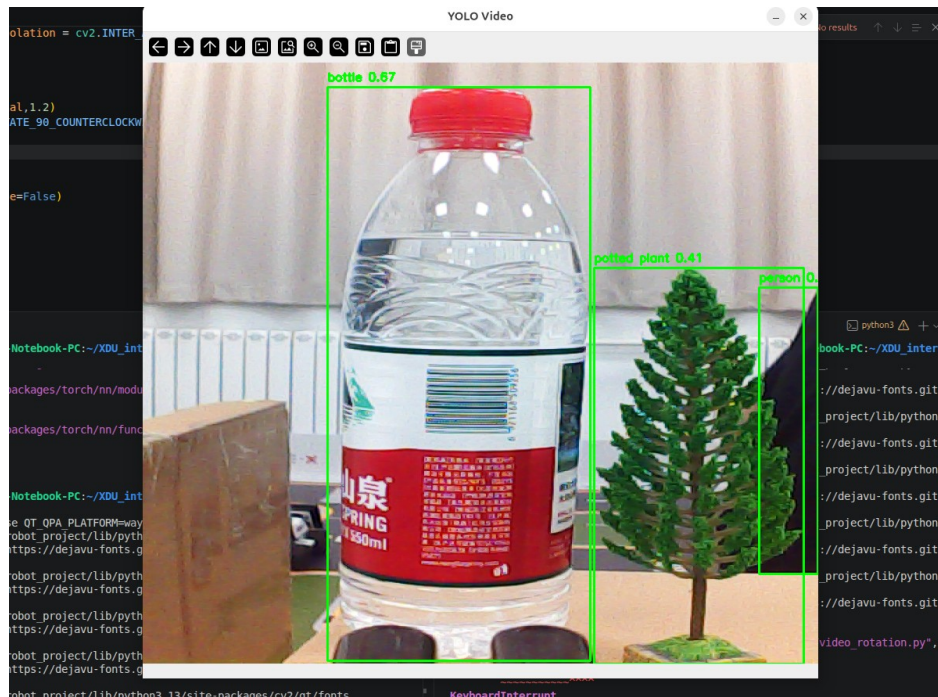


To make it more visible, will rescale the video frame by frame.

Result:



we use a model available on Ultralytic website, and it does pretty good not only vegetables recognition but all object recognition.



But for our detection to be more efficient, we need to make a personal model well trained.

container on the hard driver:

```
docker run -it \  
--name yahboom_robot \  
--privileged \  
--device /dev/ttyUSB0:/dev/ttyUSB0 \  
--device /dev/video0:/dev/video0 \  
--device /dev/video1:/dev/video1 \  
--device /dev/video2:/dev/video2 \  
--device /dev/video3:/dev/video3 \  
--env="DISPLAY=$DISPLAY" \  
--env="QT_X11_NO_MITSHM=1" \  
--volume="/tmp/.X11-unix:/tmp/.X11-unix:rw" \  
--volume="/home/kelly/rosmaster_workspace:/root/workspace" \  
ubuntu:22.04 \  
bash
```

-run the main program:

```
python3 ~/rosmaster/YahboomArm.pyc
```

- configure the virtual machine for the communication with the stm32 card:

```
stty -F /dev/ttyUSB0 57600 cs8 parenb -parodd -cstopb -echo
```

- reflash the stm32 card:

```
sudo ./stm32flash -w "/home/yahboom/Desktop/XDU_K/template.hex" -v -g 0x0 /dev/ttyUSB0
```

```
-openCv version : python3 -c "import cv2; print(cv2.__version__)"
```

-convertir .ipynb en .py:

```
1°yahboom@yahboom-virtual-machine:~/Desktop/Code/DofbotSE-Code-New/Source  
Code/Dofbot/3.ctrl_Arm$ for f in *.ipynb; do jupyter-nbconvert --to script "$f"; done
```

```
2° pip install jupyter : use to convert .ipynb into .py  
for f in *.ipynb; do jupyter --to py "$f"; done
```

```
- For opencv: pip install pyserial
```

```
- to create a symbolic link : sudo ln -s /dev/ttyUSB0 /dev/myserial
```

1- DOFBOT-JetsonNANO repository : https://www.yahboom.net/study/Dofbot-Jetson_nano

3- Tip :

<https://www.yahboom.net/public/upload/upload-html/1699442099/4.PC%20software%20control.html>

5- PC Software control:

<https://www.yahboom.net/public/upload/upload-html/1777290689/PC%20software%20control.html>

6- Dofbot-jetson-nano : https://github.com/YahboomTechnology/dofbot-jetson_nano/tree/master

7- DOFBOT SE : https://www.yahboom.net/study/DOFBOT_SE

8- DofBot manipulation :

<https://www.yahboom.net/public/upload/upload-html/1699442099/4.PC%20software%20control.html>

9- Robot arm specification: <https://www.ram-e-shop.com/shop/ro-arm-dofbot-se-ai-dofbot-se-ai-vision-robotic-arm-virtual-machine-version-6dof-degrees-of-freedom-9437>

10- Dofbot manual: <https://manuals.plus/asin/B0CT7HXYCZ>

11 – How to flash a stm32: <https://github.com/FYSETC/stm32flash#>

12- Solutions of the running of the main program: <https://manuals.plus/qa/9900005710/how-do-i-troubleshoot-the-stm32-board-on-my-yahboom-dofbot-se-robot-arm-if-it-wont-boot-and-keeps-blinking-blue-instead-of-green>

<https://manuals.plus/qa/9900005710/how-do-i-troubleshoot-the-stm32-board-on-my-yahboom-dofbot-se-robot-arm-if-it-wont-boot-and-keeps-blinking-blue-instead-of-green>

13- Driver problem : https://outbyte.com/wiki-drivers/en/du-101/?m=src_evp_adwords_du1_errors_b&content=ppcdueerrors&kw=0x80070570&gad_source=1&gad_campaignid=19784189993&gbraid=0AAAAAoMFp9aPdZUUKQThzRm3DMvf7a6_&gclid=CjwKCAjwxb7RBhA5EiwAQ-AAdBuELrHRAuelpKVa-WraYa2Ufr-VcZaLaG3QUp5P57BlDBsUVWJ5PBoCNOwQAvD_BwE

14- Fix: A Requested Power Operation is Already in Progress:

<https://www.prajwaldesai.com/a-requested-power-operation-is-already-in-progress/>

- Video rotation: <https://stackoverflow.com/questions/57827684/rotating-a-video-with-opencv-and-python-error>:
